

	AU: 2026-2027 Ateliers SOA Et Service Web Enseignante :Bènène Fradi Boumiza
Atelier 3 : Développement de service web SOAP : Banque Service	

Introduction

L'objectif de cet atelier est de développer un **service web SOAP** permettant d'exposer des fonctionnalités bancaires à travers une interface de service. À partir des classes Java et des annotations JAX-WS, le framework génère automatiquement le contrat **WSDL** et assure la transformation des objets Java en données XML échangées dans les messages SOAP. Le service sera ensuite testé et analysé avec **SoapUI**.

Objectifs

- Développer un service web SOAP avec **JAX-WS**.
- Définir une interface représentant le contrat métier du service.
- Implémenter les opérations du service.
- Utiliser les annotations JAX-WS pour exposer les opérations.
- Manipuler des objets Java dans les échanges SOAP.
- Retourner un objet Java et une collection d'objets Java à travers un service SOAP.
- Publier un service web à l'aide de Endpoint.publish().
- Consulter le contrat **WSDL** généré automatiquement.
- Identifier les opérations, paramètres et types définis dans le WSDL.
- Tester le service web avec **SoapUI**.
- Analyser les messages SOAP échangés entre le client et le service.

Outils

- Les outils nécessaires sont :
- **IntelliJ IDEA** : environnement de développement Java.
- **JDK 23** : environnement d'exécution Java.

- **Maven** : gestionnaire de dépendances et de construction du projet.
- **JAX-WS** : technologie utilisée pour développer le service SOAP.
- **SoapUI** : outil permettant de tester et d'analyser les services web SOAP.

Remarque : contrairement aux anciennes versions de Java, JAX-WS n'est plus intégré directement au JDK. Les bibliothèques JAX-WS nécessaires doivent donc être ajoutées au projet Maven.

Présentation de JAX-WS

JAX-WS (Jakarta XML Web Services) est une technologie permettant de développer et de consommer des **services web basés sur le protocole SOAP** dans l'environnement Java.

JAX-WS permet notamment de :

- exposer des méthodes Java comme des **opérations d'un service web SOAP** ;
- définir les paramètres et les résultats des opérations ;
- générer automatiquement le **contrat WSDL** à partir du service Java ;
- recevoir et traiter des **messages SOAP** ;
- convertir automatiquement les données échangées entre les messages XML et les objets Java.

Il existe deux façons pour développer un service web avec JAX-WS :

▪ Approche Top-Down — Contract First

À partir d'un document **WSDL**, les classes Java nécessaires sont générées à l'aide des outils appropriés, puis leur implémentation est complétée avant le déploiement et le test du service.

▪ Approche Bottom-Up — Code First

À partir de classes Java et d'annotations JAX-WS, le service est développé puis publié. Le framework génère automatiquement le contrat **WSDL** correspondant.

Dans cet atelier, nous adoptons une approche **Bottom-Up (Code First)** : nous commençons par créer les classes et l'interface Java du service, puis JAX-WS génère automatiquement le contrat **WSDL** correspondant.

Les annotations :

@WebService : qui déclare un service web.

- **targetNamespace**: Indique l'espace de nom XML du WSDL et des éléments XML générés à partir du service Web.
- **name**: Nom de wsdl:portType. La valeur par défaut est le nom non qualifié de la classe ou de l'interface Java.
- **serviceName**: Indique le nom du service Web :wsdl:service. La valeur par défaut est le nom simple de la classe Java + *Service*.
- **portName**: Correspond à l'attribut wsdl:portName. La valeur par défaut est `WebService.name + Port`.

@WebMethod : qui indique que c'est une opération du service web.

- **operationName**: permet de définir un nom de l'opération différent de celui défini dans la classe.
- **exclude**: permet d'exclure la méthode java du contrat WSDL généré. Sa valeur par défaut est **false**.

@WebResult : qui définit le nom du paramètre de sortie de la méthode. Par défaut il prend la valeur **return**.

@WebParam: qui définit le nom du paramètre que l'on veut avoir. Par défaut il prend la valeur **arg0**, **arg1**...

Travaux demandés

1. Créer un nouveau projet Java :Banque

- Type de projet : Maven Project
- Langage : Java
- Version du JDK : JDK 23
- Système de build : Maven

2. Configuration de Maven

Dans le fichier `pom.xml`, ajoutez les dépendances nécessaires à JAX-WS.

```
<dependencies>
  <dependency>
    <groupId>com.sun.xml.ws</groupId>
    <artifactId>jaxws-rt</artifactId>
    <version>4.0.3</version>
  </dependency>
</dependencies>
```

3. Création de l'entité Compte dans le package « entity »

Un compte set définit par :

- Un code de type long
- Un solde de type double
- La date de création de type date
- Générer un constructeur sans paramètre et un constructeur avec paramètre
- Générer les méthodes get et set

4. Création de l'interface de service BanqueService dans le package « service » contenant les opérations suivantes.

- double conversionEuroToDinar (double montant)
- Compte getCompte(long code)
- List<Compte> getComptes()

5. Création de la BanqueServiceImpl dans le package « service »

Cette classe est l'implémentation de l'interface BanqueService elle inclut les opérations qui permettent de :

- Convertir un montant de euros vers dinars

```
public double conversionEuroToDinar(double montant) {
    return montant * 3.5;}

```

- Consulter un compte et return nouveau compte
- Retourner une liste des comptes

```
public List<Compte> getComptes() {
    public Compte getCompte(long code) {
        return new Compte(code, 1000 + Math.random() * 9000, new Date());

        comptes.add(new Compte(3, 8700, new Date()));
        return comptes;
    }
}
```

Ajoutez les annotations JAX-WS suivantes avec les attributs spécifiés :

```
@WebService(
    endpointInterface = "org.sid.Service.BanqueService",
    serviceName = "BanqueService")
@WebMethod(operationName = "conversionEuroToDinar")
@WebParam(name = "montant") double montant);
@WebMethod(operationName = "getCompte")
@WebParam(name = "code") long code)
@WebMethod(operationName = "getComptes")
```

6. Créer un serveur Java

Dans la classe main ajouter :

```
String url = "http://localhost:8080/BanqueWS";

Endpoint.publish( url, new BanqueServiceImpl() );
System.out.println( "Web Service déployé sur : " + url );

System.out.println( "WSDL disponible sur : " + url + "?wsdl");
```

7. Exécuter l'application

8. Visualiser et Analyser le contrat de service : le WSDL puis :

- Identifiez le targetNamespace.
- Identifiez le service.
- Identifiez le port.
- Identifiez le binding.
- Identifiez le portType.
- Identifiez les opérations du service.
- Identifiez les paramètres de chaque opération.
- Identifiez les types XML correspondant à Compte.
- Observez comment List<Compte> est représentée dans le contrat.

9. Tester les méthodes du web service avec un analyseur SOAP : SoapUI